

Vinyl Library System — Network Protocol Documentation

Repository: [NJSNT/Java2A2](#)

Author: Jiashuo Wang(364341)

Overview

The Vinyl Library System uses a **dual-channel network architecture** combining TCP sockets for request/response communication and UDP Multicast for real-time state broadcasting to all connected clients. All messages are serialized as JSON using the Gson library. The server entry point is `VinylServer`, which listens on TCP port **2910** and delegates each client connection to a dedicated `ClientHandler` thread.

Transport Layer

Channel	Address	Port	Protocol	Purpose
TCP Socket	localhost	2910	TCP	Client sends requests; server returns ERROR or initial UPDATE_ALL
UDP Multicast	230.0.0.1	4446	UDP Multicast	Server broadcasts state changes (UPDATE_ALL) to all clients

- **Message delimiter (TCP):** newline character `\n` via `PrintWriter.println()` / `BufferedReader.readLine()`
- **UDP buffer size:** 65,536 bytes
- **Serialization:** JSON (Gson)

Message Structure

All messages use the `NetworkMessage` class serialized to JSON.

```
{
  "type":    "<Type enum value>",
  "username": "<string | null>",
```

```
"vinyl":    { <VinylDTO object> | null },
"vinylList": [ <VinylDTO>, ... ] | null,
"errorMsg": "<string | null>"
}
```

Field Reference

Field	Java Type	Required When	Description
type	NetworkMessage.Type	Always	Message type enum — never null
username	String	RESERVE, BORROW, RETURN	Operating user's name (case-insensitive matching)
vinyl	VinylDTO	ADD, REMOVE, RESERVE, BORROW, RETURN	Single vinyl record data
vinylList	List<VinylDTO>	UPDATE_ALL responses only	Full vinyl collection snapshot
errorMsg	String	ERROR responses only	Human-readable error description

VinylDTO Schema

The vinyl field carries a VinylDTO object with the following structure.

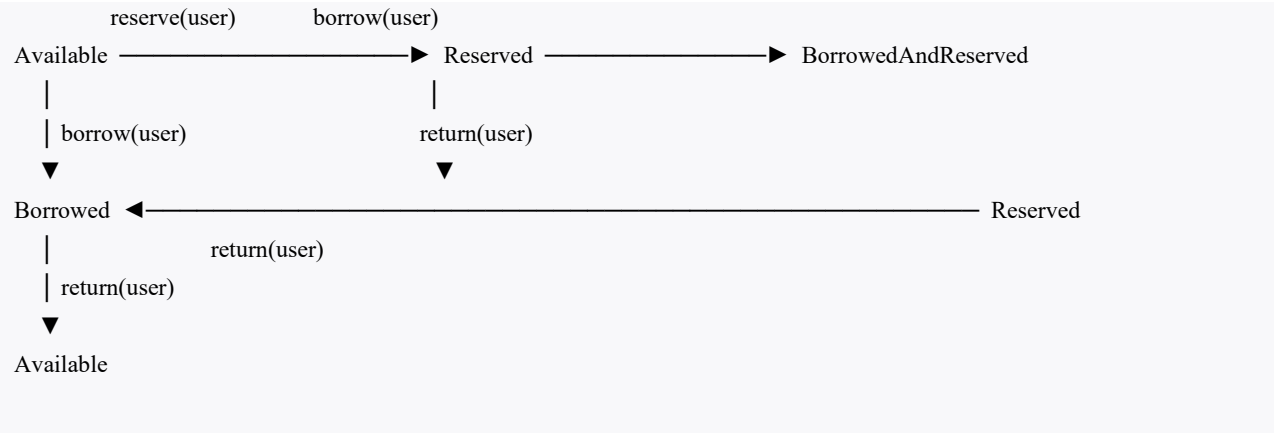
```
{
  "title":      "<string>",
  "artist":     "<string>",
  "releaseYear": <int>,
  "state":      "<Available | Reserved | Borrowed | BorrowedAndReserved>",
  "borrowingUserId": "<string | null>",
  "reservingUserId": "<string | null>",
  "markedForRemoval": <boolean>
}
```

Field	Type	Description
title	String	Vinyl title — unique key, case-insensitive
artist	String	Artist name
releaseYear	int	Release year (e.g. 1969)

state	String	Current state label (see State Machine below)
borrowingUserId	String	Username of current borrower; null if not borrowed
reservingUserId	String	Username of current reserver; null if not reserved
markedForRemoval	boolean	Whether the vinyl is flagged for removal

Vinyl State Machine

Vinyl records transition between four states implemented via the State Pattern.



State	state String	Permitted Actions
AvailableState	"Available"	BORROW, RESERVE
ReservedState	"Reserved"	BORROW (reserver only)
BorrowedState	"Borrowed"	RETURN (borrower only), RESERVE
BorrowedAndReservedState	"BorrowedAndReserved"	RETURN (borrower only)

Message Types

Type Enum Overview

Type	Direction	Channel	Description
GET_ALL	Client → Server	TCP	Request full vinyl list

ADD	Client → Server	TCP	Add a new vinyl
REMOVE	Client → Server	TCP	Mark a vinyl for removal
RESERVE	Client → Server	TCP	Reserve a vinyl
BORROW	Client → Server	TCP	Borrow a vinyl
RETURN	Client → Server	TCP	Return a borrowed vinyl
UPDATE_ALL	Server → Client	TCP or UDP Multicast	Broadcast updated vinyl list
ERROR	Server → Client	TCP	Return error details

GET_ALL

Request the complete current vinyl list from the server.

Request:

```
{ "type": "GET_ALL" }
```

Success Response (TCP unicast to requesting client):

```
{  
  "type": "UPDATE_ALL",  
  "vinylList": [ ... ]  
}
```

ADD

Add a new vinyl record to the library.

Request:

```
{  
  "type": "ADD",  
  "vinyl": {  
    "title": "Abbey Road",  
    "artist": "The Beatles",  
    "releaseYear": 1969,  
  }  
}
```

```
"state": "Available",
"borrowingUserId": null,
"reservingUserId": null,
"markedForRemoval": false
}
}
```

Validation Rules:

- vinyl must not be null
- title must not be null or blank
- artist must not be null or blank
- title must not already exist in the library (case-insensitive)

Success: Strategy returns null; Observer triggers Multicast UPDATE_ALL to all clients.

Failure: ERROR response sent via TCP to requesting client only.

Error Examples:

```
{ "type": "ERROR", "errorMsg": "Invalid request: Complete vinyl details required." }
{ "type": "ERROR", "errorMsg": "A vinyl with the title 'Abbey Road' already exists." }
```

REMOVE

Mark a vinyl for removal. The vinyl is flagged with markedForRemoval: true but remains in the list.

Request:

```
{
  "type": "REMOVE",
  "vinyl": { "title": "Abbey Road" }
}
```

Validation Rules:

- vinyl.title must not be null
- Vinyl must exist in the library

Success: Strategy returns null; Observer triggers Multicast UPDATE_ALL.

Failure: ERROR response sent via TCP.

Error Examples:

```
{ "type": "ERROR", "errorMsg": "Invalid request: No vinyl title provided." }  
{ "type": "ERROR", "errorMsg": "Vinyl not found in server library: Abbey Road" }
```

RESERVE

Reserve an available vinyl for a specific user.

Request:

```
{  
  "type": "RESERVE",  
  "username": "alice",  
  "vinyl": { "title": "Thriller" }  
}
```

Validation Rules:

- vinyl.title must not be null
- username must not be null or blank
- Vinyl must exist in the library
- Vinyl must not already be reserved (isReserved() == false)
- Vinyl must not be both markedForRemoval and not currently borrowed

Success: Strategy returns null; Observer triggers Multicast UPDATE_ALL.

Failure: ERROR response sent via TCP (with username attached).

Error Examples:

```
{ "type": "ERROR", "errorMsg": "Invalid request: No username provided.", "username": null }  
{ "type": "ERROR", "errorMsg": "Vinyl 'Thriller' is already reserved by bob.", "username": "alice" }  
{ "type": "ERROR", "errorMsg": "Vinyl 'Thriller' has been marked for removal and cannot be reserved.", "username": "alice" }
```

BORROW

Borrow a vinyl record.

Request:

```
{
  "type": "BORROW",
  "username": "alice",
  "vinyl": { "title": "Thriller" }
}
```

Validation Rules:

- `vinyl.title` must not be null
- `username` must not be null or blank
- Vinyl must exist in the library
- Vinyl must not already be borrowed (`isBorrowed() == false`)
- If vinyl is reserved, `username` must match `reservingUserId` (case-insensitive)

Success: Strategy returns null; Observer triggers Multicast UPDATE_ALL.

Failure: ERROR response sent via TCP (with username attached).

Error Examples:

```
{ "type": "ERROR", "errorMsg": "Vinyl 'Thriller' is already borrowed by alice.", "username": "bob" }
{ "type": "ERROR", "errorMsg": "Vinyl 'Thriller' is reserved by another user (alice).", "username": "bob" }
```

RETURN

Return a previously borrowed vinyl.

Request:

```
{
  "type": "RETURN",
  "username": "alice",
  "vinyl": { "title": "Thriller" }
}
```

```
}
```

Validation Rules:

- `vinyl.title` must not be null
- `username` must not be null or blank
- Vinyl must exist in the library
- Vinyl must currently be borrowed (`isBorrowed() == true`)
- `username` must match `borrowingUserId` (case-insensitive); only the borrower can return

Success: Strategy returns null; Observer triggers Multicast `UPDATE_ALL`.

Failure: ERROR response sent via TCP (with `username` attached).

Error Examples:

```
{ "type": "ERROR", "errorMsg": "Vinyl 'Thriller' is not currently borrowed.", "username": "alice" }  
{ "type": "ERROR", "errorMsg": "Unauthorized return: Only the borrower (alice) can return this vinyl.", "username": "bob" }
```

UPDATE_ALL

Sent by the server to deliver a complete snapshot of the current vinyl library.

Response:

```
{  
  "type": "UPDATE_ALL",  
  "vinylList": [  
    {  
      "title": "Abbey Road",  
      "artist": "The Beatles",  
      "releaseYear": 1969,  
      "state": "Available",  
      "borrowingUserId": null,  
      "reservingUserId": null,  
      "markedForRemoval": false  
    },  
    {  
      "title": "Thriller",  
      "artist": "Michael Jackson",  

```



```
{
  "releaseYear": 1982,
  "state": "Borrowed",
  "borrowingUserId": "alice",
  "reservingUserId": null,
  "markedForRemoval": false
}
```

Triggered by:

- Client connects (TCP unicast to new client)
- Client sends GET_ALL (TCP unicast to requesting client)
- Any state-changing operation succeeds: ADD, REMOVE, RESERVE, BORROW, RETURN (UDP Multicast to all clients)

ERROR

Sent by the server when a request fails validation or processing.

Response:

```
{
  "type": "ERROR",
  "errorMsg": "<description of the failure>",
  "username": "<username | null>"
}
```

Client Handling:

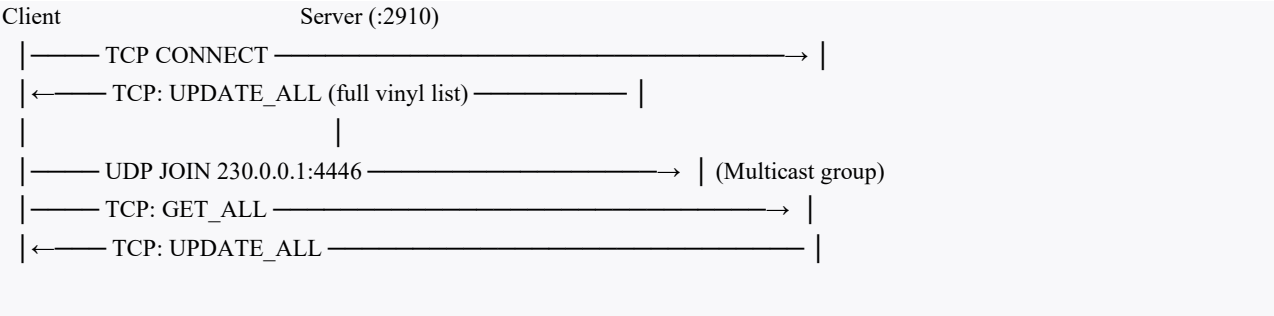
- If username is null OR equals the current session user → display error dialog to the user
- If username differs from the current session user → log silently to stderr (background simulator errors)

System-Level Errors:

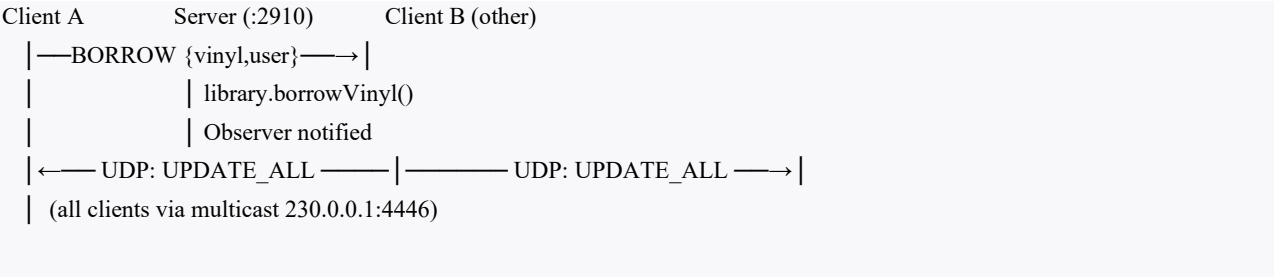
```
{ "type": "ERROR", "errorMsg": "Unknown or unsupported action type." }
{ "type": "ERROR", "errorMsg": "Server processing error: <exception message>" }
```

Communication Flow Diagrams

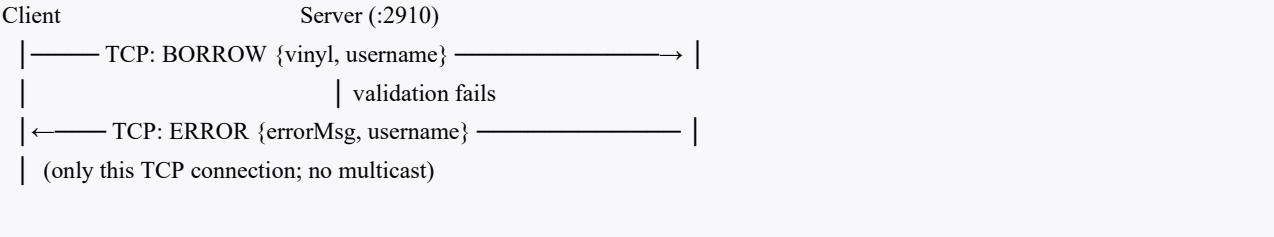
Connection Establishment



Successful State-Changing Operation (e.g. BORROW)



Failed Operation



Server-Side Routing (Strategy Pattern)

ClientHandler deserializes each incoming JSON line into a NetworkMessage, then MessageStrategyFactory performs a static HashMap lookup to route the message to the correct MessageStrategy implementation.

type	Strategy Class	Core Library Call
GET_ALL	GetAllStrategy	Returns UPDATE_ALL directly via TCP unicast
ADD	AddStrategy	library.addVinyl(title, artist, year)

REMOVE	RemoveStrategy	library.markVinylForRemoval(vinyl)
RESERVE	ReserveStrategy	library.reserveVinyl(vinyl, username)
BORROW	BorrowStrategy	library.borrowVinyl(vinyl, username)
RETURN	ReturnStrategy	library.returnVinyl(vinyl, username)
Unknown	— (null)	Returns ERROR: "Unknown or unsupported action type."

When a strategy succeeds, it returns null to ClientHandler (no direct TCP reply). The VinylLibrary notifies its Observer (MulticastStateBroadcaster), which encodes the updated list as a UPDATE_ALL JSON datagram and broadcasts it over UDP Multicast to all joined clients.

Broadcast Architecture

MulticastStateBroadcaster is registered as an Observer on VinylLibrary at server startup. Whenever the library state changes, update(Vinyl vinyl) is called, which:

1. Fetches the full current vinyl list from VinylLibrary
2. Maps each Vinyl to a VinylDTO
3. Constructs a NetworkMessage of type UPDATE_ALL
4. Serializes to JSON and sends a UDP DatagramPacket to 230.0.0.1:4446

On the client side, NetworkClient joins the same multicast group on startup and runs a dedicated daemon thread (receiveMulticastLoop) that listens for incoming UPDATE_ALL datagrams and updates the local ObservableList<Vinyl> on the JavaFX Application Thread.

Server Seed Data

The server pre-loads the following vinyl records at startup.

Title	Artist	Release Year
Abbey Road	The Beatles	1969
The Dark Side of the Moon	Pink Floyd	1973
Thriller	Michael Jackson	1982

Random Access Memories	Daft Punk	2013
Rumours	Fleetwood Mac	1977

Endpoint & Address Summary

Purpose	Address	Port	Protocol
TCP Server	localhost	2910	TCP
Multicast Broadcast	230.0.0.1	4446	UDP Multicast